

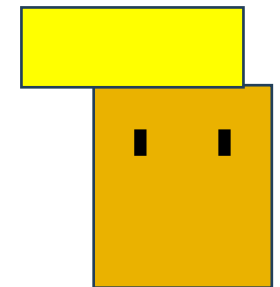
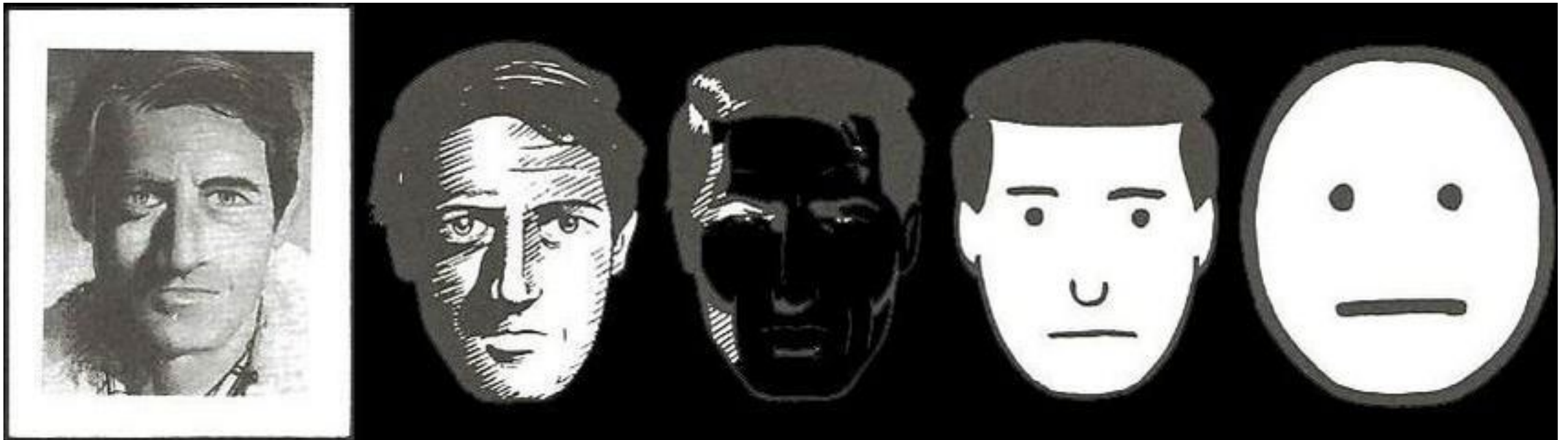
Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

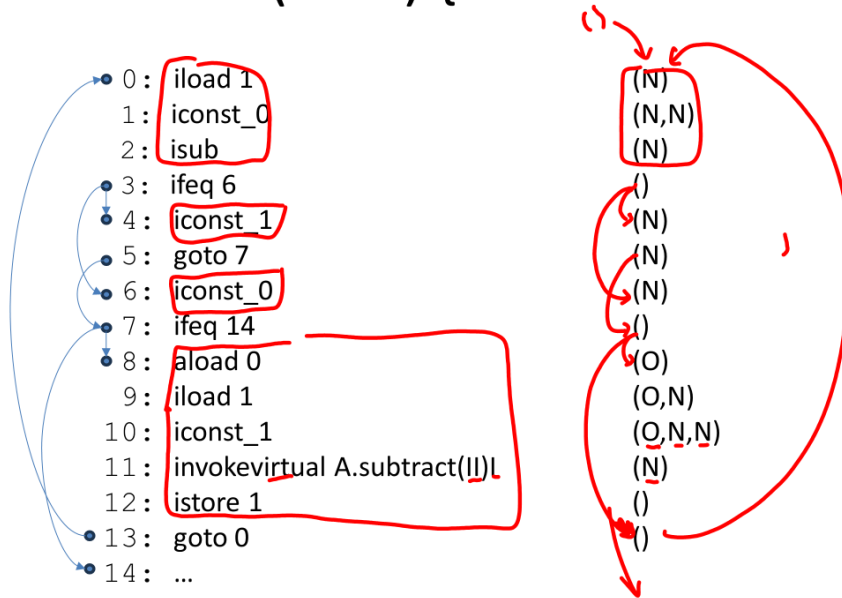
Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Abstrakte Interpretation von TRIPLA



Bildquelle: Understanding Comics: The Invisible Art, Scott McCloud, New York: Harper Collins & Kitchen Sink Press, 1993. (Seite 31)

while (x!=0) { x=this.subtract(x,1); }



Erläuterung des Beispiels im Originalpapier

"As an example, consider the process of generating SPARC instructions for the bytecodes from Figure 1. As the compiler steps through the first three bytecodes, it pushes three entries onto the compile-time stack. Figure 3 shows the contents of the stack at that point, assuming that local variables y and z have been allocated to registers L2 and L3. **When the compiler encounters the multiplication bytecode, it emits code to perform that operation.** The top two entries on the compiletime stack specify the operands. The constant integer 2 can be included as an immediate operand. A register, L0 in this case, must be selected to hold the intermediate result. The compiler then emits the instruction `smul L3, 2 -> L0` results and updates the compile-time stack, as shown in Figure 4. The compiler processes the addition in a similar manner. If local variable x was allocated to a register (for example, L1), the result of the addition can be placed directly in that register. The result in that case is the instruction `add L2, L0 -> L1`. This example demonstrates how a JIT compiler removes interpretation overhead, reducing the entire expression to as few as two SPARC instructions."

0: iload y
1: iconst 2
2: iload z
3: imul
4: iadd
5: istore x



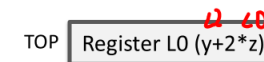
generated code

...



... `smul L3, 2 -> L0`

0: iload y
1: iconst 2
2: iload z
3: imul
4: iadd
5: istore x



... `smul L3, 2 -> L0`
`add L2, L0 -> L1`

$$\begin{aligned}
 G_{TRIPLA} = \{ & E \longrightarrow \text{let } D \text{ in } E \\
 & \quad | \text{ID} \\
 & \quad | \text{ID (A)} \\
 & \quad | E \text{ OP } E \\
 & \quad | \text{CONST} \\
 & \quad | \text{ID = E} \\
 & \quad | E ; E \\
 & \quad | \text{if } E \text{ then } E \text{ else } E \\
 & \quad | \text{while } E \text{ do } E \\
 & \quad | \text{do } E \text{ while } E \\
 & A \longrightarrow E \mid A , E \\
 & D \longrightarrow \text{ID (V) } \{ E \} \mid D D \\
 & V \longrightarrow \text{ID } \mid V , V \\
 & \text{ID} \longrightarrow \text{identifier} \\
 & \text{CONST} \longrightarrow \text{boolean or integer constant} \\
 & \text{OP} \longrightarrow \text{operators (+, -, *, /, ==, !=, <, >)} \quad \}
 \end{aligned}$$

Figure 1: Syntax of TRIPLA

Abstract Domains

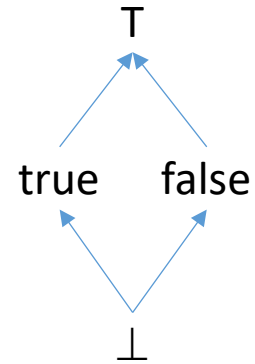
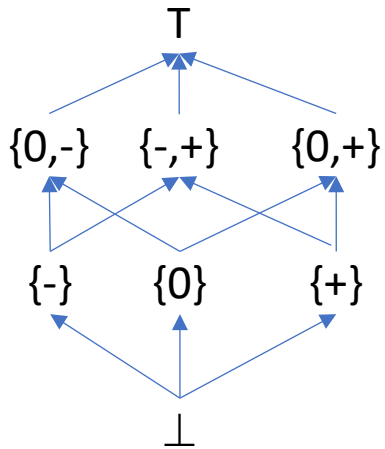
Partielle Ordnung

$$0 \sqsubseteq T \quad \perp \sqsubseteq +$$

Kleinste, obere Schranken (Least upper bounds)

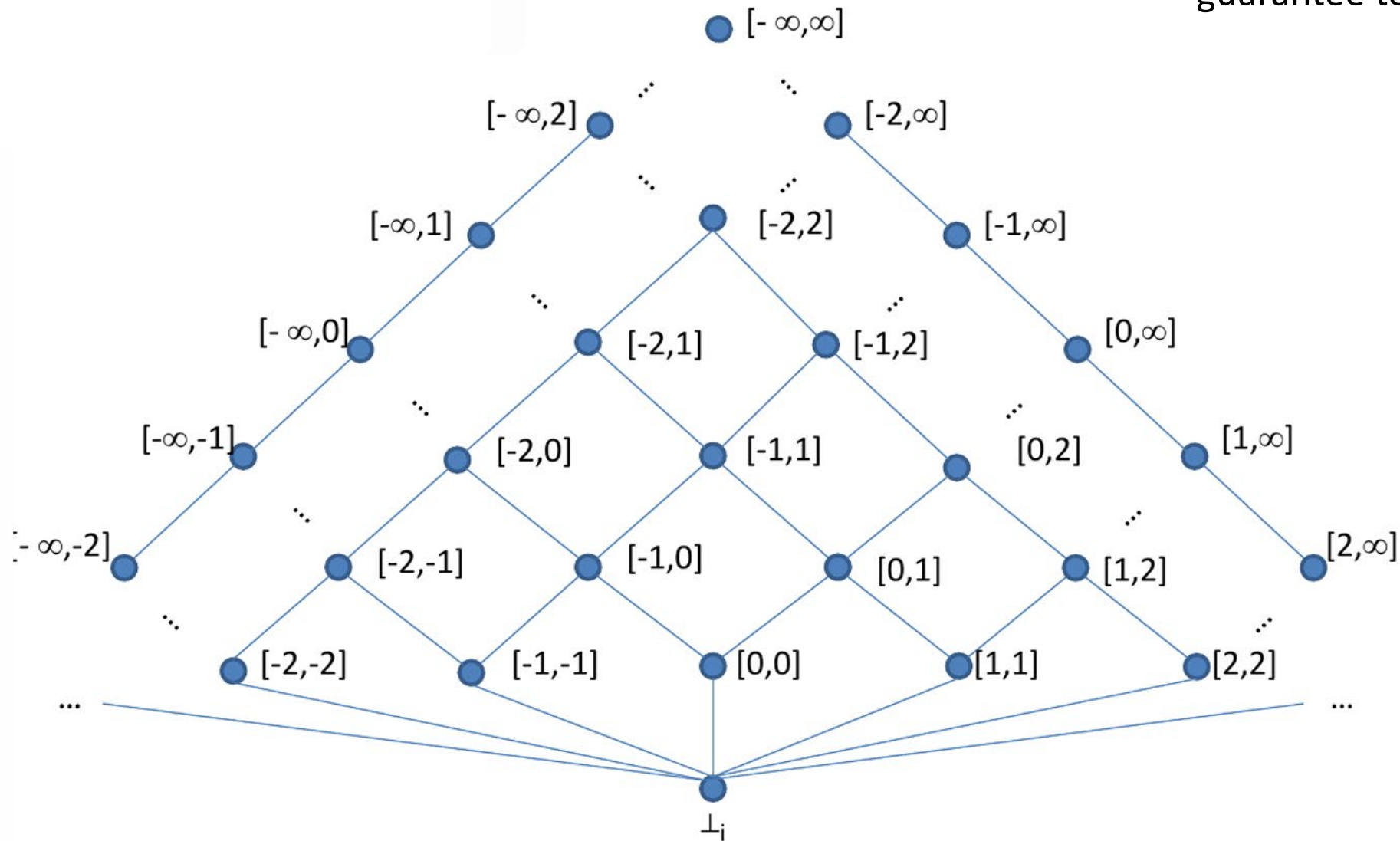
$$\perp \sqcup + = +$$

$$0 \sqcup + = T$$

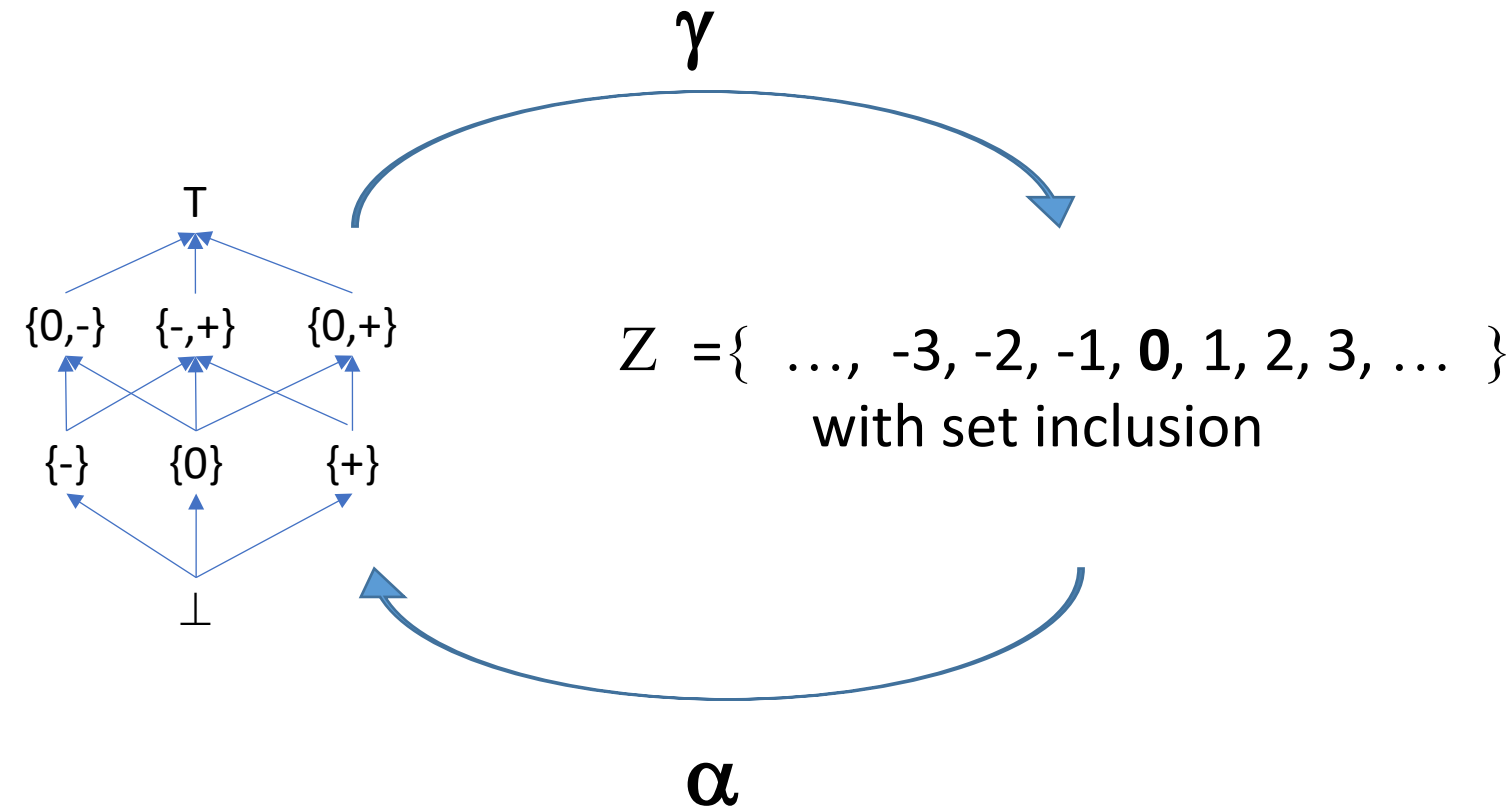


Interval Domain

Example shows infinite domain.
Finite „subsets“ can be used to
guarantee termination.



Abstraction and Concretization



$$\gamma(T) = Z$$

$$\gamma(\{-\}) = \{\dots, -3, -2, -1\}$$

$$\gamma(\{0\}) = \{0\}$$

$$\gamma(\{+\}) = \{1, 2, 3, \dots\}$$

$$\gamma(\perp) = \{\}$$

$$\gamma(\{a, b\}) = \gamma(\{a\}) \cup \gamma(\{b\})$$

$$\alpha(\{z\}) = \{+\} \quad \text{for } z > 0$$

$$\alpha(\{0\}) = \{0\} \quad \text{for } z = 0$$

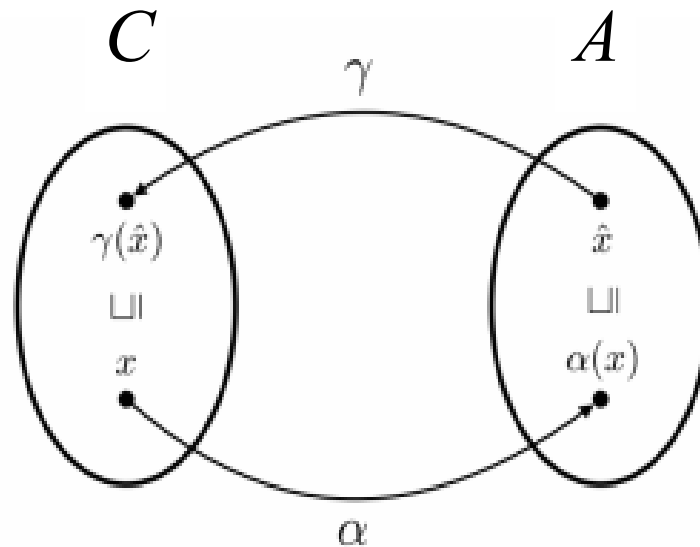
$$\alpha(\{z\}) = \{-\} \quad \text{for } z < 0$$

$$\alpha(M) = \bigcup_{z \in M} \alpha(\{z\}) \quad \text{for } |M| > 1$$

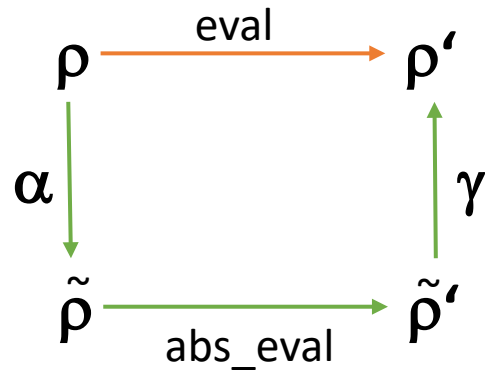
Galois Connection

In general, it is important that the abstract domain respects the ordering of the concrete domain. Thus we require that α and γ form a Galois connection

$$\forall x \in C, \forall \tilde{x} \in A : \alpha(x) \sqsubseteq \tilde{x} \Leftrightarrow x \sqsubseteq \gamma(\tilde{x})$$



Abstract and Concrete Semantics



$$\text{eval}(e, \rho) \in \gamma(\text{abs_eval}(e, \alpha(\rho), \varphi))$$

$$\alpha(\text{eval}(e, \rho)) \sqsubseteq \text{abs_eval}(e, \alpha(\rho), \varphi)$$

Simple Expressions and Assignment

$$eval(c, \rho) = (c, \rho) \text{ where } c \in \text{CONST}$$

$$eval(x, \rho) = (\rho(x), \rho) \text{ where } x \in \text{ID}$$

$$eval(e_1 \text{ op } e_2, \rho) = (\overline{op}(v_1, v_2), \rho_2) \\ \text{where } (v_1, \rho_1) = eval(e_1, \rho) \text{ and } (v_2, \rho_2) = eval(e_2, \rho_1) \\ \text{and } \overline{op} \text{ is the operation represented by } op$$

$$eval(x=e, \rho) = (v, \rho_1[x/v]) \\ \text{where } (v, \rho_1) = eval(e, \rho)$$

Abstract Operations

For each concrete mathematical or logical operation $\overline{op}$, the abstraction function induces an abstract operation

$$\tilde{op}(a_1, a_2) = \bigsqcup_{c_1 \in \gamma(a_1), c_2 \in \gamma(a_2)} \alpha(\overline{op}(c_1, c_2))$$

Abstract Operations

Note, if a_1 or a_2 are $\perp$, then the result of the abstract operation will always be $\perp$. In other words, we assume that the concrete operations are strict, i.e. if the evaluation of one of its arguments does not terminate, then the evaluation of the operation does not terminate as well. Thus, in the tables in Figure 3 we do not include these cases. The tables only show the base cases, for the remaining abstract values, the abstract results can be computed based on these cases: $\tilde{op}(A, B) = \bigsqcup_{a \in A, b \in B} \tilde{op}(a, b)$

$\tilde{+}$	-	0	+	T	$\tilde{\times}$	-	0	+	T
-	-	-	T	T	-	T	false	false	T
0	-	0	+	T	0	true	false	false	T
+	T	+	+	T	+	true	true	T	T
T	T	T	T	T	T	T	T	T	T

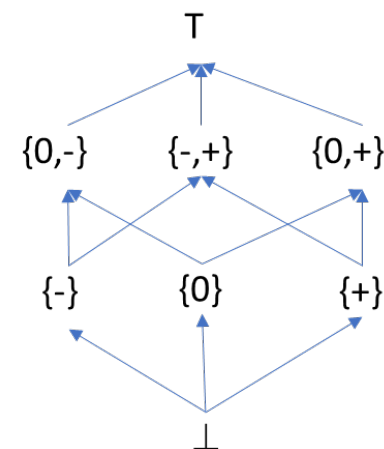


Figure 3: Abstraction operations

Simple Expressions and Assignment

$$abs_eval(c, \rho, \varphi) = (\alpha(c), \rho) \text{ where } c \in \text{CONST}$$

$$abs_eval(x, \rho, \varphi) = (\rho(x), \rho) \text{ where } x \in \text{ID}$$

$$abs_eval(e_1 \text{ op } e_2, \rho, \varphi) = (\tilde{op}(v_1, v_2), \rho_2)$$

where $(v_1, \rho_1) = abs_eval(e_1, \rho, \varphi)$ and
 $(v_2, \rho_2) = abs_eval(e_2, \rho_1, \varphi)$ and $\tilde{op}$ is the abstract
counterpart of the operation represented by op

$$abs_eval(x=e, \rho, \varphi) = (v, \rho_1[x/v])$$

where $(v, \rho_1) = abs_eval(e, \rho, \varphi)$

Abstract Environments

$$ENV : ID \cup (ID \times \mathbb{N}) \rightarrow \mathbb{Z} \cup (ID^* \times L_{G_{TRIPLA}}(E))$$

$$\widetilde{ENV} = \{\alpha(\rho) : \rho \in ENV\}$$

$\alpha(\rho)$ replaces all occurrences of a concrete value in ρ by its abstract value.

Least upper bound of two abstract environments:

$$\begin{aligned} \rho_1 \sqcup \rho_2 = & \{ \rho_1(x) \sqcup \rho_2(x) \mid x \in \text{dom}(\rho_1) \cap \text{dom}(\rho_2) \} \\ & \cup \{ \rho_1(x) \mid x \in \text{dom}(\rho_1) - \text{dom}(\rho_2) \} \\ & \cup \{ \rho_2(x) \mid x \in \text{dom}(\rho_2) - \text{dom}(\rho_1) \} \end{aligned}$$

$$\text{dom}(\rho) = \{x \mid (x, y) \in \rho\}$$

Control-Flow

$$\begin{aligned} eval(e_1; e_2, \rho) &= \begin{cases} (v_2, \rho_2) & \text{if } v_2 \neq \emptyset \\ (v_1, \rho_2) & \text{if } v_2 = \emptyset \end{cases} \\ &\quad \text{where } (v_1, \rho_1) = eval(e_1, \rho) \text{ and } (v_2, \rho_2) = eval(e_2, \rho_1) \\ \\ eval(\text{if } e_1 \text{ then } e_2 \text{ else } e_3, \rho) &= \begin{cases} eval(e_2, \rho_1) & \text{if } v_1 = \text{true} \\ eval(e_3, \rho_1) & \text{if } v_1 = \text{false} \end{cases} \\ &\quad \text{where } (v_1, \rho_1) = eval(e_1, \rho) \\ \\ eval(\text{while } e_1 \text{ do } e_2, \rho) &= \begin{cases} eval(e_2; \text{while } e_1 \text{ do } e_2, \rho_1) & \text{if } v_1 = \text{true} \\ (\emptyset, \rho_1) & \text{if } v_1 = \text{false} \end{cases} \\ &\quad \text{where } (v_1, \rho_1) = eval(e_1, \rho) \\ \\ eval(\text{do } e_1 \text{ while } e_2, \rho) &= eval(e_1; \text{while } e_2 \text{ do } e_1, \rho) \end{aligned}$$

Control-Flow

$$\begin{aligned} \text{abs_eval}(e_1; e_2, \rho, \varphi) &= \begin{cases} (v_2, \rho_2) & \text{if } v_2 \neq \emptyset \\ (v_1, \rho_2) & \text{if } v_2 = \emptyset \end{cases} \\ &\text{where } (v_1, \rho_1) = \text{abs_eval}(e_1, \rho, \varphi) \\ &\text{and } (v_2, \rho_2) = \text{abs_eval}(e_2, \rho_1, \varphi) \end{aligned}$$

$$\begin{aligned} \text{abs_eval}(\text{if } e_1 \text{ then } e_2 \text{ else } e_3, \rho, \varphi) &= \begin{cases} \text{abs_eval}(e_2, \rho_1, \varphi) & \text{if } v_1 = \text{true} \\ \text{abs_eval}(e_3, \rho_1, \varphi) & \text{if } v_1 = \text{false} \\ (v_2 \sqcup v_3, \rho_2 \sqcup \rho_3) & \text{otherwise} \end{cases} \\ &\text{where } (v_1, \rho_1) = \text{abs_eval}(e_1, \rho, \varphi) \\ &\text{and } (v_2, \rho_2) = \text{abs_eval}(e_2, \rho_1, \varphi) \\ &\text{and } (v_3, \rho_3) = \text{abs_eval}(e_3, \rho_1, \varphi) \end{aligned}$$

Control-Flow

$$abs_eval(\text{while } e_1 \text{ do } e_2, \rho, \varphi) = \begin{cases} (\emptyset, \rho_1) & \text{if } v_1 = \text{false} \\ (\tilde{v}^k, \tilde{\rho}^k) \text{ for smallest } k \text{ with} & \text{otherwise} \\ \quad (\tilde{v}^k, \tilde{\rho}^k) = (\tilde{v}^{k+1}, \tilde{\rho}^{k+1}) & \end{cases}$$

where $(v_1, \rho_1) = abs_eval(e_1, \rho, \varphi)$
and for the fixed-point computation:

$$\begin{aligned} (\tilde{v}^0, \tilde{\rho}^0) &= (\perp, \rho_1) \\ (\tilde{v}^i, \tilde{\rho}^i) &= (\tilde{v}^{i-1} \sqcup \tilde{v}_1^i, \tilde{\rho}^{i-1} \sqcup \tilde{\rho}_2^i) \end{aligned}$$

with $(\tilde{v}_1^i, \tilde{\rho}_1^i) = abs_eval(e_2, \tilde{\rho}^{i-1}, \varphi)$
and $(\tilde{v}_2^i, \tilde{\rho}_2^i) = abs_eval(e_1, \tilde{\rho}_1^i, \varphi)$

$$abs_eval(\text{do } e_1 \text{ while } e_2, \rho, \varphi) = abs_eval(e_1; \text{while } e_2 \text{ do } e_1, \rho, \varphi)$$

$v' = \text{None}$
 $g' = \text{None}$

$v = \perp$
 $g = g_1$

while $(v \neq v') \&\& (g \neq g')$

$\left\{ \begin{array}{l} v' = v \\ g' = g \end{array} \right.$

$(v, g'') = \text{absval}(e_2, g, \varphi)$

$(v'', g) = \text{absval}(e_1, g'', \varphi)$

$v = v \sqcup v'$
 $g = g \sqcup g'$



$\left. \begin{array}{l} v = v \sqcup v' \\ g = g \sqcup g' \end{array} \right\}$

Fixpunkt-Berechnung der **while**-Schleife in Pseudo-Code

Hinweis: Im Pseudo-Code sind die Variablen Programmvariablen und nicht Variablen im mathematischen Sinn!

Beispiel

```
/* Simple while loop
```

```
rho = { "x" : "+", "y" : "+" } --> abs_eval = ('+', {'x': 'T', 'y': '+'})
```

```
rho = { "x" : "+", "y" : "-" } --> abs_eval = ('T', {'x': 'T', 'y': 'T'})
```

```
rho = { "x" : "-", "y" : "+" } --> abs_eval = (None, {'x': '-', 'y': '+'})
```

```
rho = { "x" : "-", "y" : "-" } --> abs_eval = (None, {'x': '-', 'y': '-'})
```

```
rho = { "x" : "0+", "y" : "+" } --> abs_eval = ('+', {'x': 'T', 'y': '+'})
```

```
*/
```

```
while (x>0) do {
```

```
    x=x-y;
```

```
    y=y+1
```

```
}
```

Functions

overwrite

restore

Big Step
Semantics

$$\begin{aligned}
 eval(\text{let } f_1(x_{1,1}, \dots, x_{1,k_1}) \{e_1\} &= (v, \rho^*) \\
 \vdots &\text{ where } \rho' = \rho[(f_1, k_1) / ((x_{1,1}, \dots, x_{1,k_1}), e_1)] \\
 f_n(x_{n,1}, \dots, x_{n,k_n}) \{e_n\} &\vdots \\
 \text{in } e, &[(f_n, k_n) / ((x_{n,1}, \dots, x_{n,k_n}), e_n)] \\
 \rho) &\text{ and } (v, \rho'') = eval(e, \rho') \\
 &\text{ and } \rho^* = \rho''[(f_1, k_1) / \rho(f_1, k_1)] \dots [(f_n, k_n) / \rho(f_n, k_n)]
 \end{aligned}$$

$$\begin{aligned}
 eval(f(e_1, \dots, e_n), \rho) &= (v, \rho^*) \\
 &\text{ where } \rho(f, n) = ((x_1, \dots, x_n), e) \\
 &\text{ and } (v_1, \rho_1) = eval(e_1, \rho), \\
 &\quad (v_2, \rho_2) = eval(e_2, \rho_1), \\
 &\quad \vdots \\
 &\quad (v_n, \rho_n) = eval(e_n, \rho_{n-1}) \\
 &\text{ and } \rho' = \rho_n[x_1/v_1] \dots [x_n/v_n] \\
 &\text{ and } (v, \rho'') = eval(e, \rho') \\
 &\text{ and } \rho^* = \rho''[x_1/\rho_n(x_1)] \dots [x_n/\rho_n(x_n)]
 \end{aligned}$$

Functions

$$\begin{aligned}
 \text{abs_eval}(\text{let } f_1(x_{1,1}, \dots, x_{1,k_1})\{e_1\} &= (v, \rho^*) \\
 \vdots &\text{ where } \rho' = \rho[(f_1, k_1)/((x_{1,1}, \dots, x_{1,k_1}), e_1)] \\
 f_n(x_{n,1}, \dots, x_{n,k_n})\{e_n\} &\vdots \\
 \text{in } e, \rho, \varphi) &[(f_n, k_n)/((x_{n,1}, \dots, x_{n,k_n}), e_n)] \\
 &\text{and } \varphi' = \varphi - \bigcup_{1 \leq i \leq n} (\{f_i\} \times \widetilde{\mathbb{Z}}^{k_i} \rightarrow \widetilde{ENV}) \\
 &\text{and } (v, \rho'') = \text{abs_eval}(e, \rho', \varphi') \\
 &\text{and } \rho^* = \rho''[(f_1, k_1)/\rho(f_1, k_1)] \cdots [(f_n, k_n)/\rho(f_n, k_n)]
 \end{aligned}$$

$$\begin{aligned}
 \text{abs_eval}(f(e_1, \dots, e_n), \rho, \varphi) &= (v, \rho^*) \\
 \text{where } \rho(f, n) &= ((x_1, \dots, x_n), e) \\
 \text{and } (v_1, \rho_1) &= \text{abs_eval}(e_1, \rho, \varphi), (v_2, \rho_2) = \text{abs_eval}(e_2, \rho_1, \varphi), \dots, (v_n, \rho_n) = \text{abs_eval}(e_n, \rho_{n-1}, \varphi) \\
 \text{and } \rho' &= \rho_n[x_1/v_1] \cdots [x_n/v_n] \\
 \text{and } (v, \rho'') &= \begin{cases} \text{abs_eval}(e, \rho', \varphi[(f, v_1, \dots, v_n)/\rho']) & \text{if } \varphi(f, v_1, \dots, v_n) \text{ is undefined} \\ (\perp, \rho') & \text{if } \varphi(f, v_1, \dots, v_n) = \tilde{\rho} \text{ and } \tilde{\rho} = \rho' \\ \text{abs_eval}(e, \tilde{\rho} \sqcup \rho', \varphi[(f, v_1, \dots, v_n)/\tilde{\rho} \sqcup \rho']) & \text{if } \varphi(f, v_1, \dots, v_n) = \tilde{\rho} \text{ and } \tilde{\rho} \neq \rho' \end{cases} \\
 \text{and } \rho^* &= \rho''[x_1/\rho_n(x_1)] \cdots [x_n/\rho_n(x_n)]
 \end{aligned}$$

Functions

$$\begin{aligned}
 &abs_eval(\text{let } f_1(x_{1,1}, \dots, x_{1,k_1})\{e_1\} \quad = \quad (v, \rho^*) \\
 &\quad \vdots \quad \text{where } \rho' = \rho[(f_1, k_1)/((x_{1,1}, \dots, x_{1,k_1}), e_1)] \\
 &\quad f_n(x_{n,1}, \dots, x_{n,k_n})\{e_n\} \quad \vdots \\
 &\text{in } e, \rho, \varphi) \quad [(f_n, k_n)/((x_{n,1}, \dots, x_{n,k_n}), e_n)] \\
 &\quad \text{and } \varphi' = \varphi - \bigcup_{1 \leq i \leq n} (\{f_i\} \times \widetilde{\mathbb{Z}}^{k_i} \rightarrow \widetilde{ENV}) \\
 &\quad \text{and } (v, \rho'') = abs_eval(e, \rho', \varphi') \\
 &\quad \text{and } \rho^* = \rho''[(f_1, k_1)/\rho(f_1, k_1)] \cdots [(f_n, k_n)/\rho(f_n, k_n)]
 \end{aligned}$$

Lösche neu
definierte
Funktionen
im Cache

Functions

$$\begin{aligned}
 &abs_eval(f(e_1, \dots, e_n), \rho, \varphi) = (v, \rho^*) \\
 &\text{where } \rho(f, n) = ((x_1, \dots, x_n), e) \\
 &\text{and } (v_1, \rho_1) = abs_eval(e_1, \rho, \varphi), (v_2, \rho_2) = abs_eval(e_2, \rho_1, \varphi), \dots, (v_n, \rho_n) = abs_eval(e_n, \rho_{n-1}, \varphi) \\
 &\text{and } \rho' = \rho_n[x_1/v_1] \cdots [x_n/v_n] \\
 &\text{and } (v, \rho'') = \begin{cases} abs_eval(e, \rho', \varphi[(f, v_1, \dots, v_n)/\rho']) & \text{if } \varphi(f, v_1, \dots, v_n) \text{ is undefined} \\ (\perp, \rho') & \text{if } \varphi(f, v_1, \dots, v_n) = \tilde{\rho} \text{ and } \tilde{\rho} = \rho' \\ abs_eval(e, \tilde{\rho} \sqcup \rho', \varphi[(f, v_1, \dots, v_n)/\tilde{\rho} \sqcup \rho']) & \text{if } \varphi(f, v_1, \dots, v_n) = \tilde{\rho} \text{ and } \tilde{\rho} \neq \rho' \end{cases} \\
 &\text{and } \rho^* = \rho''[x_1/\rho_n(x_1)] \cdots [x_n/\rho_n(x_n)]
 \end{aligned}$$

Funktionsaufruf in
Cache eintragen

Der Funktionsaufruf wurde

- noch nicht
- bereits mit gleicher Bindungsumgebung
- bereits mit unterschiedlicher Bindungsumgebung im Cache eingetragen.

Beispiele

```
/* Non-recursive function with loop
```

```
rho = { "i0" : "+", "limit0" : "0", "step0" : "+" } --> abs_eval = ('+', {'i0': '+', 'limit0': '0', 'step0': '+'})  
rho = { "i0" : "-", "limit0" : "-", "step0" : "-" } --> abs_eval = ('-', {'i0': '-', 'limit0': '-', 'step0': '-'})  
rho = { "i0" : "-", "limit0" : "+", "step0" : "-" } --> abs_eval = (None, {'i0': '-', 'limit0': '+', 'step0': '-'})
```

```
*/
```

```
let f(i,limit,step) {  
    while (i>limit) do { i = i + step }  
}  
in f(i0,limit0,step0)
```

Beispiele

/* Recursive function with loop

```
rho = { "i0" : "+", "limit0" : "0", "step0" : "+" }
--> abs_eval = ('bot', {'i0': '+', 'limit0': '0', 'step0': '+'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "-" }
--> abs_eval = ('-', {'i0': '-', 'limit0': '-', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "+", "step0" : "-" }
--> abs_eval = ('-', {'i0': '-', 'limit0': '+', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "+" }
--> abs_eval = ('T', {'i0': '-', 'limit0': '-', 'step0': '+'})
```

"bot" in the first case indicates that the recursion does not terminate

*/

```
let f(i,limit,step) {
  if (i>limit) then i = i + step; f(i, limit, step) else i
}
in f(i0,limit0,step0)
```

/* Recursive non-terminating function

```
rho = { "i0" : "+", "limit0" : "0", "step0" : "+" }
--> abs_eval = ('bot', {'i0': '+', 'limit0': '0', 'step0': '+'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "-" }
--> abs_eval = ('bot', {'i0': '-', 'limit0': '-', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "+", "step0" : "-" }
--> abs_eval = ('bot', {'i0': '-', 'limit0': '+', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "+" }
--> abs_eval = ('bot', {'i0': '-', 'limit0': '-', 'step0': '+'})
```

"bot" indicates that the recursion does not terminate

*/

```
let f(i,limit,step) {
  f(i, limit, step)
}
in f(i0,limit0,step0)
```

/* Mutually recursive function

```
rho = { "i0" : "+", "limit0" : "0", "step0" : "+" }
--> abs_eval = ('bot', {'i0': '+', 'limit0': '0', 'step0': '+'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "-" }
--> abs_eval = ('-', {'i0': '-', 'limit0': '-', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "+", "step0" : "-" }
--> abs_eval = ('-', {'i0': '-', 'limit0': '+', 'step0': '-'})
rho = { "i0" : "-", "limit0" : "-", "step0" : "+" }
--> abs_eval = ('T', {'i0': '-', 'limit0': '-', 'step0': '+'})
```

"bot" in the first case indicates that the recursion does not terminate

*/

```
let f1(i,limit,step) {
  if (i>limit) then
    i = i + step; f2(i, limit, step)
  else i }

f2(i,limit,step) {
  if (i>limit) then
    i = i + step; f1(i, limit, step)
  else i }
in f2(i0,limit0,step0)
```